

Tyler Harris
599 Edge AI and Robotics
Module 1 Parallel Programming

Overview:

The objective of this lab is to get familiar with the NVIDIA CUDA platform and its integration with Python. It covered libraries including PyCUDA, Numba, and CuPy. The lab demonstrates how to offload intensive tasks from the CPU to the GPU.

Setup:

The lab provides 4 jupyter notebooks containing the code for demonstration as well as a few sections to implement small functions using the libraries. A Docker container is provided to simplify the process.

Notebook 1: Introduction to CUDA and PyCUDA

Overview:

The purpose of this lab is to introduce the core concepts of the CUDA parallel computing platform through the PyCUDA library. The lab demonstrates memory transfer, creating CUDA kernels, and the use of abstractions like `gpuarray` and `ElementwiseKernel` to simplify tasks.

CUDA overview:

CUDA allows developers to use NVIDIA GPUs for general purpose processing. Host is the CPU and its associated memory. Device is the GPU and its dedicated memory (VRAM). A kernel is a function written in CUDA that is executed by GPU threads.

Exercises:

1) Device Query and Initialization

- a) The goal of this exercise was to understand the hardware constraints and capabilities of the Jetson Nano GPU. A CUDA driver is initialized and queried by the hardware. The lab examines key metrics such as the compute capability and total VRAM.

2) Manual Memory Management

- a) This exercise demonstrated the transfer of data from the host and device.
- b) Memory is allocated via `cuda.mem_alloc`, which allocates space on the GPU
- c) Transfer is performed (from host to device) using `cuda.memcpy_htod`, this moves the array to the previously allocated memory.
- d) The provided kernel is compiled and executed
- e) Transfer is again performed (device to host) using `cuda.memcpy_dtoh`, this brings the results back to the CPU

3) GPUArray Abstraction

- a) This portion demonstrated using PyCUDA wrappers to reduce boilerplate code

- b) The same exercise is repeated using `pycuda.gpuarray`, this showed how PyCUDA can abstract the explicit memory transfers and allocations. This makes GPU arrays appear more similar to standard arrays when executing operations.

4) Arithmetic Operations and Memory Handlers

- a) The goal of this portion was to demonstrate how vector addition and matrix operations can be performed using memory handlers.
- b) Kernels are provided for 1D array addition and 2D matrix addition. Using [`cuda.In\(\)`](#), `cuda.Out()`, and `cuda.InOut()` arguments. These automate the memory allocation and transfer processes which greatly simplifies the code.

5) Elementwise Kernels and Linear Combinations

- a) This exercise focused on multi stage expressions without multiple kernel launches
- b) Using `ElementwiseKernel` to calculate a linear combination, it was demonstrated that multiple mathematical steps can be performed in a single pass over the data.

6) Interoperability with Numba

- a) This portion focused on exploring the CUDA Array Interface to share data between different GPU libraries.
- b) By using `pycuda.autoprimaryctx`, both PyCUDA and Numba are able to share the same GPU context. An array is created in PyCUDA and processed using Numba `@cuda.jit` kernel.

7) Element-wise Squaring

- a) The last part of the lab was a small exercise to use the `gpuarray` class and perform elementwise squaring.

Exercise

Perform element-wise squaring of numbers in a matrix using the `gpuarray` class.

```
# Write your own code here
import pycuda.gpuarray as gpuarray
import numpy as np

# create matrix (host)
h_matrix = np.array([[1, 2], [3, 4]], dtype=np.float32)

# transfer
d_matrix = gpuarray.to_gpu(h_matrix)

# square
d_result = d_matrix ** 2

# get result
print("Original:\n", h_matrix)
print("Squared (GPU):\n", d_result.get())
```

```
Original:
[[1.  2.]
 [3.  4.]]
Squared (GPU):
[[ 1.  4.]
 [ 9. 16.]]
```

Notebook 2: Introduction to CUDA Python with Numba

Overview:

This lab introduces Numba which is a JIT compiler designed for Python code. It allows for Python's ease of use with the performance of C or CUDA by compiling specialized machine code for specific data types.

Exercises:

- 1) The lab initially covers the basic imports needed, from the numba library the lab uses `jit`, `vectorize`, `guvectorize`, and `cuda`.
- 2) The next step demonstrates how to compile functions on the CPU, this is performed using decorators on python functions. In this initial example the basic `@jit` decorator is used.
- 3) The lab then covers how to benchmark the performance difference between the functions. Using the `%timeit` function and the `.py_func` attribute to access the uncompiled function, it is simple to view the speedup.

```
%timeit monte_carlo_pi.py_func(nsamples)
```

```
2.37 ms ± 16.2 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

```
%timeit monte_carlo_pi(nsamples)
```

```
84.2 µs ± 4.59 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

- 4) The next step of the lab goes more in depth on compilation options.
 - a) Initially it demonstrates that Numba can not compile all Python code. This is because certain types such as dictionaries cannot be efficiently compiled. Attempting to attach a Numba decorator to a function with non translatable types will give a warning and types are not specialized.
 - b) Numba has an object mode which does not type specialize, however using the `nopython` mode will achieve higher performance. With `nopython` set it will throw an error since it cannot infer the type.
 - c) You may also use the `.inspect_types()` function to view the results of type inference.
- 5) The lab then covers Universal Functions (ufuncs). By using the `@vectorize` decorator with the target set to `cuda`, numba transforms a scalar function into a parallelized operation that broadcasts across entire arrays. This is very simple to implement but has the overhead cost of memory transfers.

- 6) Building upon ufuncs, the lab introduces generalized ufuncs (gufuncs). By using the `@guvectorize` decorator functions can handle sub-arrays or rows of input such as calculating the sum of every row in a matrix, rather than just element wise operations.
- 7) The last decorator introduced is the `@cuda.jit` tag. These follow the CUDA execution model and require defining the number of threads and blocks per grid, performing thread indexing, and it demonstrates striding to process larger datasets.
- 8) Next the lab covers a memory management tutorial. Data transfer has a large impact on performance and the lab highlights some techniques to transfer memory and save on performance.
 - a) `cuda.to_device(obj)` is used to manually move data to the GPU and it persists across multiple function calls.
 - b) `cuda.device_array_like(obj)` allocates memory on the GPU without initializing it which avoids excess transfers.
 - c) `copy_to_host()` brings the results back to the CPU
- 9) Next there is a more in depth look at writing CUDA kernels. The lab demonstrates again using the `@cuda.jit` decorator and how to define grid and block size, index threads, and perform a stride. It is then showed how to call the function by first specifying grid and block size.
- 10) The last portion of the lab is a number of exercises, both Numba basics and modifying/writing CUDA kernels. The code and output for these exercises is included in the Jupyter notebook on GitHub.

Notebook 3: Array summation and matrix multiplication

Overview:

This notebook serves as a tutorial for performing traditional computing operations such as array summation and matrix multiplication on the CPU compared to the GPU.

Exercises:

- 1) Array Summation Comparison
 - a) Using the CPU, a for loop iterates through the range of an array, adding the elements one by one. This execution is serial and as the array grows in size the time increases linearly.
 - b) Using the GPU (parallel), a CUDA kernel is used to perform the same operation. On the GPU every addition happens across the 2048 threads (16 blocks x 128 threads). `cuda.synchronize()` is used to ensure all threads are finished before transferring results to the CPU.
- 2) Matrix Multiplication: Striding
 - a) Matrix multiplication is very expensive. Instead of assigning a thread to each element, threads use a for loop to stride across the data. Once a thread finishes calculating it jumps forward to process the next available element. This allows for the kernel to process matrices of any size despite the physical number of threads available.
- 3) Matrix Multiplication: Shared Memory

- a) Another performance optimization is using shared memory. Standard GPU memory is slow and if every thread must read from global memory the performance drops significantly.
- b) Matrices are broken into smaller tiles, all threads work together to load a tile from global memory into shared memory (`cuda.shared.array`).
- c) Threads are then synchronized to ensure all threads finish loading the tile before the calculation starts. Threads now may use the faster shared memory cache to perform the computation.

Notebook 4: Benchmarking Speed: NumPy vs. CuPy

Overview:

This notebook focuses on performance best practices by comparing NumPy, the industry-standard CPU library for scientific computing, with CuPy, an open-source array library designed for GPU-accelerated computing. The goal is to demonstrate how swapping to a CUDA-backed library can provide speedups of over 100x for large datasets.

Exercises:

- 1) Array Creation and Random Initialization
 - a) Generating large-scale data (e.g. 10,000 x 10,000 matrices) is the first step in benchmarking.
 - b) NumPy: Utilizes the CPU to generate random numbers. For matrices of this scale, the CPU must process the request through system memory, which can be relatively slow.
 - c) CuPy: Utilizes the GPU and the `cuRAND` library. This allows for massive parallelism in number generation.
- 2) Trigonometric Operations
 - a) This exercise tests complex element-wise operations, specifically calculating the identity $\sin^2(x) + \cos^2(x) = 1$ across every element in the array.
 - b) Trigonometric functions are mathematically intensive. While the CPU must iterate through these calculations, the GPU treats each array element as an independent task. For a 100 million element array ($10,000^2$), CuPy significantly outperforms NumPy because the overhead of launching the GPU kernel is negligible compared to the massive amount of computation performed.
- 3) Arithmetic and Compound Operations
 - a) Benchmarking simple scalar multiplication (`x *= 10`) and array-on-array multiplication.
 - b) Broadcasting: Both libraries support broadcasting, but CuPy utilizes the GPU's high memory bandwidth and thousands of cores to apply these changes. Scaling: When performing multiple operations in a row (e.g., multiplying two arrays after scaling them), the GPU's advantage compounds. The more operations performed while the data remains in Device Memory, the more the cost of the initial data transfer is amortized.
- 4) Summary of Performance Best Practices

- a) Library Compatibility: CuPy's interface is designed to be highly compatible with NumPy, making it easy to migrate existing codebases to the GPU by simply swapping the library prefix.
- b) Kernel Fusion: CuPy can sometimes "fuse" multiple operations into a single GPU pass, reducing the number of times data must be read from and written to the GPU's global memory.
- c) Thresholds: It is important to note that for very small arrays, NumPy may still be faster due to the lack of GPU launch overhead. GPU acceleration is most effective once the data reaches a scale where the parallel processing power outweighs the communication latency.